

Navigation and Wheel Commands (robot_control)

Navigation and Wheel Commands

How `robot_control/` turns a navigation goal into wheel speeds, including the wavefront planner that feeds it.

The chain at a glance

```
NavigationController      state machine: rotates, then follows path
|
├→ WavefrontPathPlanner  Dijkstra + gradient descent → list of waypoints
|
├→ FollowPathController  Pure Pursuit + CTE-PD → curvature  $\kappa$ 
|
├→ Diff-drive            $l = \text{base}(1 - \kappa \cdot W/2), \quad r = \text{base}(1 + \kappa \cdot W/2)$ 
|
├→ Deadband boost + clamp  $\pm 1$ 
|
├→ Action(left_speed, right_speed)  $\in [-1, 1]^2$ 
|
├→ env.apply():
|   - SimEnv: scales by max_wheel_speed → ICC physics
|   - RealEnv: ActionSender → MicromvpAdapter → SerialActionSender → UD
```

All numbers below are from `config/controller.yaml` and `core/types.py`: `max_speed = 0.3`, `wheel_base = 30 cm`, `car_width = 8`, `car_height = 10`, so `car_size = 10 cm`.

1. NavigationController state machine

File: `controller/navigation.py:110`.

```
IDLE → PRE_ROTATING → FOLLOWING → POST_ROTATING → FINISHED
      |               |
      (skip if aligned) (skip if no goal-θ)
```

Entry: `navigate_to(goal_x, goal_y, goal_theta, current_pos, obstacles)`:

1. Call the injected `PathPlanner.plan(start, goal, obstacles)` (WavefrontPathPlanner by default, or RVGPlanner).
2. Filter near-duplicate waypoints (min spacing `0.1 × car_size = 1 cm`).
3. Hand the path to `FollowPathController.set_path(path)` — resampled at ≤ `0.01 × car_size = 0.1 mm` spacing,

prefix arc length built.

4. Set state to `PRE_ROTATING` with target = angle to first non-trivial waypoint.

After that, every tick goes through `step(obs, ...)`.

2. PRE_ROTATING (rotate to face the path)

`navigation.py:412` → `_calculate_rotation_action`.

Variable-speed band based on heading error magnitude:

```
lerrl > 45°      speed = rotation_speed_max = 0.25
15° < lerrl ≤ 45° speed = lerp(0.15, 0.25, (lerrl-15)/30)
lerrl ≤ 15°      speed = rotation_speed_min = 0.15
```

Direction (CCW positive):

```
err > 0    → left = -speed, right = +speed    (CCW: left back, right forward)
err < 0    → left = +speed, right = -speed    (CW)
```

Skip-condition: if at entry `lerrl < pre_rotation_skip_angle = 45°`, jump directly to FOLLOWING.

Stable-hold termination: must stay inside `rotation_tolerance_deg = 2.5°` for `rotation_stable_time = 0.5 s` continuously → transition to FOLLOWING. The timer kills oscillation around the setpoint.

3. FOLLOWING — `FollowPathController._calculate_action`

File: `controller/follow_path.py:323`. This is where the actual curvature is computed.

3a. Goal check

```
dist_to_goal = ||robot - path[-1]||
if dist_to_goal < goal_tolerance:      # 0.2 · car_size = 2 cm
    return Action.stop()
```

3b. Find target point (with arc-length gating)

`_find_target_point` (line 505). Search window: `[path_index, path_index + no_skip_ratio · car_size]` along arc length. With `no_skip_ratio = 0.5` and `car_size = 10 cm` → 5 cm window. In that window, pick the first point whose distance from robot ≥ `lookahead_distance = 0.5 · car_size = 5 cm`. If none qualifies, pick the last point in the window. The gating prevents skipping forward at path self-intersections.

3c. Find closest reference (for CTE)

`_find_closest_reference` (line 561): project robot onto each segment in the gated window. The closest projection gives `θ_ref` (segment heading) and the signed cross-track error:

```
n = (-sin θ_ref, cos θ_ref)          # left-of-segment normal
cte = n · (robot - ref_point)        # positive = robot is left of path
```

3d. Pure Pursuit curvature

`_pure_pursuit_curvature` (line 539):

```
heading_err = wrap(atan2(target.y - robot.y, target.x - robot.x) - robot_θ)
L           = max(distance_to_target, lookahead_distance) # never below 5 cm
κ_pp        = 2 · sin(heading_err) / L                    # units: 1/cm
```

Geometric meaning: the unique circular arc starting at the robot tangent to its heading and ending at the target.

3e. Cross-Track-Error PD (only when close to path)

```
use_pd = (lcte ≤ 2.5 · car_size = 25 cm)
```

If not close enough → PD disengaged, `κ_pd = 0`, PD state reset (no windup).

If close:

```
cte_used = cte
if lcte_used < cte_deadband (0.1·car_size = 1 cm): cte_used = 0
cte_used = clamp(cte_used, ±cte_clip = 1·car_size = 10 cm)

# Numerical derivative + EMA smoothing
ḡte_raw = (cte_used - prev_cte) / dt
ḡte_filt = 0.75 · ḡte_filt + 0.25 · ḡte_raw          # α = 0.25

# Normalize, then convert to curvature units
cte_n    = cte_used / car_size
ḡte_n    = ḡte_filt / car_size
κ_pd     = (1.2 · cte_n + 0.35 · ḡte_n) / car_size    # Kp=1.2, Kd=0.35
κ_pd     = clamp(κ_pd, ±curv_pd_max = 2.0/car_size = 0.2 /cm)
```

The double divide by `car_size` is dimensional: first normalize CTE to a dimensionless ratio, then divide the gain output again to land in 1/cm. The whole controller is geometry-invariant — change `car_size` and the controller retunes itself.

3f. Combine

```
κ_cmd = clamp(κ_pp + κ_pd, ±curvature_max = 3.5/car_size = 0.35 /cm)
```

3g. Speed scheduling

Two cuts:

```
# Cut 1: large heading error → slow down
```

```

turn_factor = max(0.25, cos(heading_err))
base_speed  = max_speed * turn_factor          # max_speed = 0.3

# Cut 2: high commanded curvature → slow down
curv_slow   = clamp(1 - 0.65 * |κ_cmd|/κ_max, 0.35, 1.0)
base_speed *= curv_slow

```

Net effective speed range: roughly `[0.026, 0.3]` of full wheel scale.

3h. Mode override: ALIGN (rotate-in-place hysteresis)

Before any of the curvature math runs (line 367):

```

if currently ALIGN:      exit when |heading_err| ≤ 45°
if not ALIGN:            enter when |heading_err| ≥ 60°

```

In ALIGN, rotate in place at `w = 0.6 * max_speed = 0.18`:

```

left  = -w if heading_err > 0 else +w
right = +w if heading_err > 0 else -w

```

A second rotate-in-place case (line 461): `|heading_err| > 110°` and PD disengaged (far from path).

4. Curvature → wheel speeds (differential drive)

`follow_path.py:478`:

```

diff      = κ_cmd * wheel_base / 2          # wheel_base = 30 cm
left_speed = base_speed * (1 - diff)
right_speed = base_speed * (1 + diff)

```

Derivation. For diff-drive at linear velocity `v` and angular velocity `ω`:

```

v_left  = v - ω·W/2
v_right = v + ω·W/2

```

Curvature `κ = ω/v`, so `ω·W/2 = v·κ·W/2 = v·diff`. Substituting `v = base_speed`:

```

v_left  = base * (1 - diff)
v_right = base * (1 + diff)

```

`base_speed` is the linear forward speed; `diff` is the wheel asymmetry that produces the requested curvature.

Then normalize so the larger wheel does not exceed `max_speed` (preserves ratio → preserves curvature when saturating):

```

max_wheel = max(l1l, lrl)
if max_wheel > max_speed:
    l *= max_speed / max_wheel
    r *= max_speed / max_wheel

```

5. Deadband scaling

`_enforce_deadband_scale` (`follow_path.py:27`):

```

m = max(l1l, lrl)
if m < 1e-6:      return (0, 0)          # truly stop
if m ≥ 0.05:      return (l, r)          # above deadband, no change
scale = 0.05 / m
l, r = l.scale, r.scale          # boost so larger wheel hits 0.05
if max(l1l, lrl) > 1.0:  re-normalize    # safety

```

Real motors below ~5% stall and produce no motion. A commanded `(0.02, 0.01)` would just freeze; scaling to `(0.10, 0.05)` preserves the ratio (so curvature is preserved) and gets both wheels turning.

A final `clamp(±1)` produces `Action(left_speed, right_speed)`.

6. POST_ROTATING

After `FollowPathController.is_done()` reports `FINISHED`, if a `goal_θ` was specified, `NavigationController` switches to `POST_ROTATING` — same in-place rotation primitive as Section 2, targeting the goal heading. Then `FINISHED` and `Action.stop()`.

7. What happens to `Action` after `env.apply()`

Simulation (`environment/sim.py:235`)

```

v_left  = action.left_speed  · max_wheel_speed    # units: cm/s
v_right = action.right_speed · max_wheel_speed

```

Then `_simulate_step` (line 27) integrates exact ICC (Instantaneous Center of Curvature) kinematics:

```

v      = (v_right + v_left) / 2                # linear velocity
ω      = (v_right - v_left) / wheel_base        # angular velocity

if ω ≈ 0:                                       # straight motion
    new_x = x + v·dt·cos(θ)
    new_y = y + v·dt·sin(θ)
    new_θ = θ

```

```

else:
    # arc motion
    r = v / w # turning radius
    new_theta = theta + w*dt
    new_x = x + r*(sin(new_theta) - sin(theta))
    new_y = y - r*(cos(new_theta) - cos(theta))

```

This is the inverse of the kinematics in Section 4 — sim and controller are self-consistent.

Real robot (`environment/real.py:162` → `micromvp_adapter.py:105`)

```

RealEnv.apply(Action)
→ sender.send(robot_id, action.left_speed, action.right_speed)
→ MicromvpAdapter.send(...)
→ SerialActionSender (from micromvp lib)
→ UDP packet to robot at configured IP/port
  containing {robot_id, left, right} normalized in [-1, 1]
→ Robot firmware: scales to PWM, drives motors

```

No further scaling in `robot_control` — the same `[-1, 1]` floats reach the motor controller.

8. Numerical example

Robot at `(20, 20)`, heading `30°`. Target waypoint `(28, 24)` (5 cm distance, 26.6° from robot's pose).

- `heading_err = atan2(4, 8) - 30° = 26.6° - 30° = -3.4°` → `-0.059 rad`.
- `L = max(distance, lookahead) = max(8.94, 5) = 8.94 cm`.
- `κ_pp = 2 * sin(-0.059) / 8.94 = -0.0132 /cm`.
- Suppose `cte = -0.5 cm` (robot 0.5 cm right of path). `lctel < 1 cm` deadband → `cte_used = 0`, so `κ_pd = 0`.
- `κ_cmd = -0.0132 /cm`.
- `turn_factor = max(0.25, cos(-0.059)) = 0.998` → `base = 0.3 * 0.998 ≈ 0.2995`. `curv_slow = clamp(1 - 0.65 * 0.0132/0.35, 0.35, 1.0) ≈ 0.976` → `base ≈ 0.292`.
- `diff = -0.0132 * 30 / 2 = -0.198`.
- `left = 0.292 * (1 - (-0.198)) = 0.292 * 1.198 = 0.350`. `right = 0.292 * (1 + (-0.198)) = 0.292 * 0.802 = 0.234`.
- `max_wheel = 0.350 > max_speed = 0.3` → scale by `0.3/0.350 = 0.857`. `left = 0.300, right = 0.201`.
- Both above deadband 0.05 → no boost.
- Emit `Action(left_speed=0.300, right_speed=0.201)`.

Robot turns slightly left (left wheel faster) while moving forward — corrects the small heading error toward `(28, 24)`.

9. The wavefront path planner

`controller.yaml` selects `navigation.planner: "wavefront"` (the default; alternative is `RVGPlanner`). Both expose the

same interface: `plan(start, goal, obstacles) → [(x, y), ...]` in cm.

Two layers share the "wavefront" name:

Layer	File	Purpose
<code>WavefrontPathPlanner</code> (wrapper)	<code>planner/wavefront_path_planner.py</code>	Path source for <code>NavigationController</code>
<code>WavefrontPlanner</code> (grid primitive)	<code>utils/wavefront.py</code>	Grid build, BFS, Dijkstra, gradient descent

9a. Wrapper flow — `WavefrontPathPlanner.plan` (`wavefront_path_planner.py:71`)

1. Convert obstacles: `(x_cm, y_cm, θ°, width_cm, depth_cm) → (x_m, y_m, half_x_m, half_y_m, θ°)`. `depth` is along heading (local X), `width` is perpendicular (local Y).
2. Build a fresh grid every call with `WavefrontConfig(resolution=0.005, robot_radius=car_size/2/100, inflation_margin=0)`.
3. Recover blocked start — if start cell is OBSTACLE (e.g., robot inside its own inflated footprint after a push), call `find_nearest_free(start)` and use that.
4. Plan — Dijkstra from goal → distance grid → gradient descent from start → grid path.
5. Return path in cm.
6. Debug snapshots — dumped to `output_dump/wavefront/wavefront_<timestamp>_<idx>.png` if `wavefront_debug_dir` is set.

9b. Grid build — `WavefrontPlanner.build_grid` (`utils/wavefront.py:54`)

Cell size: 5 mm (`resolution = 0.005 m`). The grid is `width × height` cells covering workspace bounds. Each cell is `FREE (0)` or `OBSTACLE (1)`.

9c. Obstacle inflation — `_add_box_to_grid` (line 98)

For each obstacle, inflate half-dimensions by `robot_radius + inflation_margin` (treats robot as point in grid). Take the AABB of the rotated, inflated rectangle. For each cell in that AABB, transform the cell center into the box's local frame:

```
lx = dx · cosθ + dy · sinθ
ly = -dx · sinθ + dy · cosθ
```

Mark OBSTACLE if `|lx| ≤ hw_inflated AND |ly| ≤ hd_inflated`. This is exact rotated-rectangle inflation — not a bounding-circle approximation.

Boundary walls: the outer `ceil(inflation/res)` cells on each edge are marked OBSTACLE so the robot is not planned within `robot_radius` of the workspace boundary.

9d. Proximity cost grid — `_build_cost_grid` (line 156)

This is what makes paths look smooth and centered rather than scraping obstacle edges. The algorithm:

1. Find **boundary cells** = OBSTACLE cells with at least one FREE 4-neighbor.
2. Multi-source BFS outward into FREE cells: `dist_cells[i,j]` = cell distance to nearest obstacle boundary, capped at `max_cells = ceil(prox_dist/res)`.
3. Cost multiplier per FREE cell: `frac = 1 - dist_cells/(max_cells + 1)` (1.0 at boundary, 0 at `prox_dist`); `cost = 1.0 + weight * frac`.

With `prox_dist = 2 cm` and `weight = 5`: a cell adjacent to an obstacle costs 6× to traverse vs a cell far from any obstacle. Cost decays linearly to 1.0 at 2 cm distance.

These cells are still FREE — just expensive. Dijkstra naturally routes through the middle of corridors.

9e. Dijkstra from goal — `_dijkstra_from_goal` (line 490)

8-connected, with proximity-weighted costs:

```
neighbors:          cardinal: cost 1.0      diagonal: cost  $\sqrt{2} \approx 1.414$   
step into (ni,nj):  step_cost = (1.0 or 1.414) * cost_grid[nj, ni]
```

Cost is applied at the destination cell, so a step ending next to an obstacle is penalized. Output: `dist[j, i]` = minimum cost from cell `(i, j)` to goal. Unreachable cells: `+∞`.

9f. Gradient descent — `_gradient_descent` (line 532)

From the start cell, at each step pick the 8-neighbor with smallest `dist` value, append to path. Stop on goal or when no neighbor improves (stuck → return empty). Safety net: `max_steps = width × height`.

9g. What the follower consumes

Raw output is a dense polyline of cell centers (5 mm apart) — even a 30 cm path has ~60 points.

`NavigationController.navigate_to` filters duplicate points (min `0.1 * car_size = 1 cm` apart) — drops most cell-step waypoints (`navigation.py:234`). `FollowPathController.set_path` then resamples to `0.01 * car_size = 0.1 mm` spacing — so the path stays dense for arc-length gating.

10. Why this design works

- **Cheap to rebuild every call.** ~9000 cells (45 × 65 cm at 0.5 cm) → BFS + Dijkstra in tens of milliseconds. Builds fresh per `plan()` — fine because the scene changes every push.
- **Soft margin without hard inflation.** Hard 2 cm inflation would close many corridors; soft cost lets the planner squeeze through tight spaces only when needed.
- **Start-blocked recovery built in.** After a push the robot is often inside its own inflated footprint — `find_nearest_free` reseats the start.
- **No local minima.** Dijkstra-from-goal + gradient descent on the cost-to-go field is monotonic by construction.
- **PD is gated by distance to path.** Far from the path, only Pure Pursuit acquires; PD would windup.
- **Normalize → deadband → clamp, in that order.** Preserves curvature even when speeds saturate or fall below the motor stall threshold.

- All thresholds scale with `car_size`. Change robot geometry and the controller retunes itself.
- ICC sim physics matches the controller's curvature model exactly. No sim-side discretization error inside the sim.
- Drop-in planner swap. RVGPlanner satisfies the same `PathPlanner` Protocol — wavefront vs RVG is a config flip.